

DSA Graphs

Complete Pattern Notes
Self-Study & Last-Minute Revision Guide

Graph Patterns — Overview

Pattern	Core Idea	Key Signal Words
BFS/DFS Hard Variants	Graph = tree + visited set. Mark visited to prevent cycles.	islands, connected, reachability, clone, word ladder
Cycle Detection	Undirected: visited non-parent. Directed: 3-color (GRAY=on stack).	detect cycle, can finish, valid tree
Topological Sort	Kahn's BFS (in-degree 0 first) or DFS post-order (add after neighbors).	prerequisites, dependencies, ordering
Union-Find	Track connected components. Find+Union with path compression+rank.	merge groups, redundant edge, accounts merge
Dijkstra	Greedy: always process node with smallest distance. Min-heap.	shortest path, minimum cost, weighted graph
Bellman-Ford	Relax all edges V-1 times. Handles negative weights.	negative weights, K stops, negative cycle
MST (Kruskal/Prim)	Kruskal: sort edges + Union-Find. Prim: grow from node + min-heap.	connect all nodes, minimum spanning tree

Key insight: Graphs = Trees without parent-child constraint. Every tree algorithm has a graph equivalent — just add a visited set.

Pattern 1 — BFS / DFS Hard Variants

Core Idea: Graph traversal = tree traversal + visited set. The visited set prevents infinite loops in cycles.

Graph Representations

```
// Adjacency List (most common)
vector<vector<int>> adj(n);
adj[u].push_back(v); adj[v].push_back(u); // undirected
// Build from edge list
for(auto& edge : edges){
    adj[edge[0]].push_back(edge[1]);
    adj[edge[1]].push_back(edge[0]);
}
```

Grid vs Graph Problems

	Grid Problems	Graph Problems
Input	2D array	Edge list or adjacency list
Neighbors	4 directions (up/down/left/right)	Adjacency list lookup
Mark visited	Modify cell value OR bool 2D array	bool visited[] array
Common pattern	Multi-source BFS, flood fill	Component counting, shortest path

Key Tricks

- Modify grid to mark visited: grid[r][c]='0' instead of separate visited array — saves space
- Reverse thinking (Pacific Atlantic): BFS from destination uphill instead of forward from source
- Implicit graph (Word Ladder): try all 26 letters at each position instead of precomputing adjacency list
- Clone graph: store clone BEFORE recursing into neighbors — handles cycles in DFS

Word Ladder — Implicit Graph BFS

```
// Try all 26 letters at each position — O(26*L) per word
for(int i=0; i<word.size(); i++){
    char orig = word[i];
    for(char c='a'; c<='z'; c++){
        word[i] = c;
        if(wordSet.count(word)){
            q.push(word);
            wordSet.erase(word); // erase = mark visited
        }
    }
    word[i] = orig; // restore original character
}
```

}

Why erase from wordSet: prevents revisiting same word. Cleaner than separate visited set. Same word twice = infinite loop.

Solved Problems

Problem	LC#	Key Insight
Number of Islands	200	DFS from each unvisited '1'. Count starts. Visited array preserves original.
Clone Graph	133	Hashmap original->clone. Store clone BEFORE processing neighbors (handles cycles).
Word Ladder	127	BFS for shortest path. Try all 26 letters implicitly. Erase from wordSet = visited.
Pacific Atlantic Flow	417	Reverse BFS from ocean borders uphill. neighbor >= current height. Intersect both sets.

Pattern 2 — Cycle Detection

Core Idea: A cycle exists when you visit a node already on the CURRENT path — not just any visited node.

Undirected vs Directed — Key Difference

	Undirected Graph	Directed Graph
Cycle signal	Visit visited neighbor that is NOT direct parent	Visit node currently on DFS stack (GRAY)
Extra tracking	Parent node	Recursion stack / color array
Algorithm	DFS with parent check	DFS with 3-color or Kahn's BFS

Directed Graph — 3-Color DFS

```
// Colors: 0=WHITE(unvisited), 1=GRAY(on stack), 2=BLACK(done)
bool hasCycle(int node, vector<vector<int>>& adj, vector<int>& color){
    color[node] = 1;    // GRAY – currently processing
    for(int neighbor : adj[node]){
        if(color[neighbor] == 1) return true;    // back edge = cycle
        if(color[neighbor] == 0)
            if(hasCycle(neighbor, adj, color)) return true;
    }
    color[node] = 2;    // BLACK – fully processed
    return false;
}
```

Why 3 colors not 2: A BLACK node is reachable from multiple paths — that's NOT a cycle. Only GRAY (currently on active stack) means cycle. 2-color gives false positives.

Undirected Graph — Parent Check DFS

```
bool hasCycle(int node, int parent, vector<vector<int>>& adj, vector<bool>& visited){
    visited[node] = true;
    for(int neighbor : adj[node]){
        if(!visited[neighbor]){
            if(hasCycle(neighbor, node, adj, visited)) return true;
        }
        else if(neighbor != parent) return true;    // visited non-parent = cycle
    }
    return false;
}
```

Solved Problems

Problem	LC#	Key Insight
Course Schedule	207	Directed graph cycle detection. 3-color DFS. GRAY node = back edge = cycle.
Graph Valid Tree	261	$n-1$ edges + all nodes connected = valid tree. Check both conditions.

Pattern 3 — Topological Sort

Core Idea: Order nodes such that for every edge $u \rightarrow v$, u comes before v . Only works on DAGs. If cycle exists, topological sort is impossible.

Two Algorithms

	Kahn's BFS	DFS Post-order
Approach	Process in-degree 0 nodes first, reduce neighbors	Add node AFTER all neighbors processed
Cycle detection	Natural: if processed $\neq n$, cycle exists	Need separate check
Level info	BFS levels = parallel processing groups	Not available
Preferred when	Need cycle detection OR level/parallel info	Just need ordering

Kahn's BFS Template

```
// Step 1: compute in-degrees
vector<int> inDegree(n, 0);
for(int u=0; u<n; u++) for(int v : adj[u]) inDegree[v]++;

// Step 2: start with in-degree 0 nodes
queue<int> q;
for(int i=0; i<n; i++) if(inDegree[i]==0) q.push(i);

// Step 3: process
vector<int> result;
while(!q.empty()){
    int node=q.front(); q.pop();
    result.push_back(node);
    for(int neighbor : adj[node]){
        inDegree[neighbor]--;
        if(inDegree[neighbor]==0) q.push(neighbor);
    }
}

// if result.size() != n -> cycle exists
```

Parallel Courses — BFS Level Counting

Each BFS level = one semester/time unit. All nodes at same level can be processed simultaneously.

```
while(!q.empty()){
    int size=q.size();
    semesters++;
    while(size--){ // process entire current level
        int node=q.front(); q.pop();
        processed++;
    }
}
```

```

        for(int neighbor : adj[node])
            if(--inDegree[neighbor]==0) q.push(neighbor);
    }
}
return processed==n ? semesters : -1;

```

Solved Problems

Problem	LC#	Key Insight
Course Schedule	207	Cycle detection via Kahn's: if processed != n, cycle exists.
Course Schedule II	210	Return topological order. Empty vector if cycle (processed != n).
Parallel Courses	1136	Count BFS levels = minimum semesters. Premium.

Pattern 4 — Union-Find (Disjoint Set Union)

Core Idea: Track connected components. Find = which group? Union = merge two groups. With path compression + union by rank: nearly $O(1)$ per operation.

Full Template with Both Optimizations

```
vector<int> parent, rank;
UnionFind(int n) : parent(n), rank(n,0){
    iota(parent.begin(), parent.end(), 0); // parent[i]=i
}
int find(int x){
    if(parent[x]!=x)
        parent[x]=find(parent[x]); // PATH COMPRESSION
    return parent[x];
}
bool unite(int x, int y){
    int px=find(x), py=find(y);
    if(px==py) return false; // already connected
    if(rank[px]<rank[py]) swap(px,py); // UNION BY RANK
    parent[py]=px;
    if(rank[px]==rank[py]) rank[px]++;
    return true; // successfully merged
}
```

Path compression: after find(x), every node from x to root points directly to root. Next find is $O(1)$. Union by rank: attach shorter tree under taller — prevents degenerate linked list.

Component Counting Pattern

```
int components = n; // start with n components
for each edge (u,v):
    if(unite(u, v))
        components--; // merged two components
return components;
```

Union-Find vs BFS/DFS

	Union-Find	BFS/DFS
Static connectivity	Both work	Both work
Dynamic connectivity	Best choice	Recompute each time $O(V+E)$
Many queries	$O(a(n))$ per query	$O(V+E)$ per query
Shortest path	Cannot do	BFS works

	Union-Find	BFS/DFS
Cycle detection	Natural byproduct	DFS works

2D Grid to 1D Index

```
index = row * numCols + col    // converts grid position to single UF index
```

Solved Problems

Problem	LC#	Key Insight
Number of Provinces	547	Start with n components. Decrement on each successful union.
Redundant Connection	684	First edge where unite() returns false = creates cycle = redundant.
Accounts Merge	721	Union accounts sharing emails. Group all emails by root. Sort emails.

Pattern 5 — Shortest Path Algorithms

Core Idea: BFS for unweighted. Dijkstra for weighted non-negative. Bellman-Ford for negative weights. Choose based on graph properties.

Algorithm Selection

Situation	Algorithm	Time
Unweighted graph	BFS	$O(V+E)$
Weighted, non-negative	Dijkstra	$O((V+E) \log V)$
Weighted, with negatives	Bellman-Ford	$O(V \cdot E)$
0 or 1 weights only	0-1 BFS	$O(V+E)$
At most K stops	Bellman-Ford K+1 iterations	$O(K \cdot E)$

Dijkstra Template

```
vector<int> dist(n, INT_MAX);
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> > pq;
dist[src]=0; pq.push({0, src});
while(!pq.empty()){
    auto [d, u]=pq.top(); pq.pop();
    if(d > dist[u]) continue; // stale entry - skip (lazy deletion)
    for(auto [v, w] : adj[u]){
        if(dist[u]+w < dist[v]){
            dist[v]=dist[u]+w;
            pq.push({dist[v], v});
        }
    }
}
```

Why if(d > dist[u]) continue: C++ priority_queue has no decrease-key. Push new entry with better distance, skip old stale entries when popped.

Bellman-Ford Template

```
vector<int> dist(n, INT_MAX);
dist[src]=0;
for(int i=0; i<n-1; i++){ // relax V-1 times
    for(auto& edge : edges){
        int u=edge[0], v=edge[1], w=edge[2];
        if(dist[u]!=INT_MAX && dist[u]+w < dist[v])
            dist[v]=dist[u]+w;
    }
}
```

```
// K stops variant: run exactly K+1 iterations with temp copy
for(int i=0; i<=k; i++){
    vector<int> temp=dist;           // copy prevents using same-iteration updates
    for each edge: relax using dist, update temp
    dist=temp;
}
```

Why temp copy for K-stops: each iteration should only use distances from PREVIOUS iteration. Without copy, a path could use more than K+1 edges in one iteration.

Custom Cost Functions

Problem Type	Cost Function	LC#
Standard shortest path	dist[u] + weight	743
Minimum effort path	max(currentEffort, abs(height diff))	1631
Cheapest with K stops	Bellman-Ford K+1 iterations	787

Solved Problems

Problem	LC#	Key Insight
Network Delay Time	743	Dijkstra from k. Answer = max of all shortest paths. -1 if any INT_MAX.
Path With Minimum Effort	1631	Dijkstra. Cost = max(current, edge diff). Return when reaching destination.
Cheapest Flights K Stops	787	Bellman-Ford K+1 iterations. temp copy prevents using same-iteration updates.

Pattern 6 — Minimum Spanning Tree

Core Idea: Subset of edges connecting ALL nodes with MINIMUM total weight. Exactly $V-1$ edges, no cycles. Two algorithms: Kruskal's (sort + Union-Find) and Prim's (grow from node + heap).

Kruskal's vs Prim's

	Kruskal's	Prim's
Approach	Sort ALL edges, add cheapest non-cycle	Grow tree from one node, add cheapest neighbor
Data Structure	Sorted edges + Union-Find	Min-heap
Best for	Sparse graphs (few edges)	Dense graphs (many edges)
Time	$O(E \log E)$	$O(E \log V)$ with heap, $O(V^2)$ with array

Kruskal's Template

```
// Step 1: build and sort all edges {weight, u, v}
sort(edges.begin(), edges.end());

// Step 2: Union-Find
int totalCost=0, edgesUsed=0;
for(auto& [w, u, v] : edges){
    if(unite(u, v)){ // returns false if already connected
        totalCost += w;
        if(++edgesUsed == n-1) break; // MST has exactly n-1 edges
    }
}
return edgesUsed==n-1 ? totalCost : -1; // -1 if disconnected
```

Prim's Template

```
vector<bool> inMST(n, false);
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
pq.push({0, 0}); // start from node 0 with cost 0
int totalWeight=0, nodesAdded=0;
while(!pq.empty()){
    auto [w, u]=pq.top(); pq.pop();
    if(inMST[u]) continue; // already in MST
    inMST[u]=true;
    totalWeight+=w; nodesAdded++;
    for(auto [v, weight] : adj[u])
        if(!inMST[v]) pq.push({weight, v});
}
return nodesAdded==n ? totalWeight : -1;
```

For dense complete graphs (like Min Cost Connect Points): Prim's with array $O(V^2)$ beats Kruskal's $O(E \log E)$ since $E = O(V^2)$.

Solved Problems

Problem	LC#	Key Insight
Min Cost Connect All Points	1584	MST of complete graph. Manhattan distance = edge weight. Prim's $O(V^2)$ or Kruskal's $O(V^2 \log V)$.

Quick Reference — All 6 Graph Patterns

Pattern	Time	Space	Use When
BFS/DFS	$O(V+E)$	$O(V)$	Traversal, components, reachability, shortest unweighted
Cycle Detection	$O(V+E)$	$O(V)$	Detect cycle. Directed: 3-color. Undirected: parent check.
Topological Sort	$O(V+E)$	$O(V)$	DAG ordering, prerequisites, parallel task scheduling
Union-Find	$O(a(n))$ per op	$O(V)$	Dynamic connectivity, component merge, redundant edges
Dijkstra	$O((V+E)\log V)$	$O(V)$	Shortest path, non-negative weights
Bellman-Ford	$O(V \cdot E)$	$O(V)$	Negative weights, K stops, negative cycle detection
MST Kruskal	$O(E \log E)$	$O(V)$	Minimum spanning tree, sparse graphs
MST Prim	$O(E \log V)$	$O(V)$	Minimum spanning tree, dense graphs

Most Important Rules

- Graph = tree + visited set. Always add visited check to prevent infinite loops in cycles.
- Directed cycle: 3-color DFS. GRAY=on stack. BLACK=done. Only GRAY neighbor = cycle.
- Topological sort: if Kahn's processed count $\neq n$, cycle exists. BFS levels = parallel processing.
- Union-Find: path compression + union by rank = nearly $O(1)$. Always use BOTH optimizations.
- `Unite()` returns false = already connected = redundant edge. Use this for cycle detection and component counting.
- Dijkstra: if($d > \text{dist}[u]$) continue — skip stale heap entries (lazy deletion).
- Bellman-Ford K stops: use temp copy of dist each iteration — prevents using same-iteration updates.
- MST: exactly $V-1$ edges. Kruskal = sort + UF. Prim = grow from node. Dense graph = Prim's.

— End of Notes —